

Java Programming 3

Week 2



Agenda this week

Last week - Project Review

3-Layer Architecture

Loose Coupling With Interfaces

Dependency Injection

@Annotations

Spring Framework: The Spring Container

Component Scanning - Autowiring

Last week - Project Review

- Topics last week: recap
- What is the state of the projects?
 - Problems?
 - What went ok?
 - What went wrong?
- Did you think about *the technical design*?
 - What package structure did you use?
 - What classes did you add?





Agenda this week

Last week - Project Review

3-Layer Architecture

Loose Coupling With Interfaces

Dependency Injection

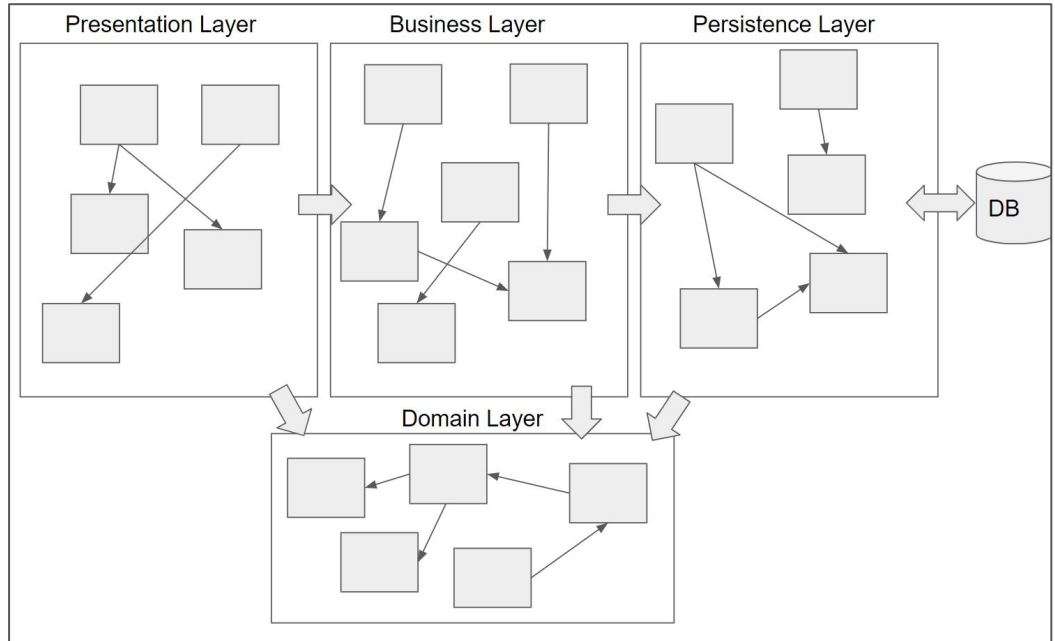
@Annotations

Spring Framework: The Spring Container

Component Scanning - Autowiring

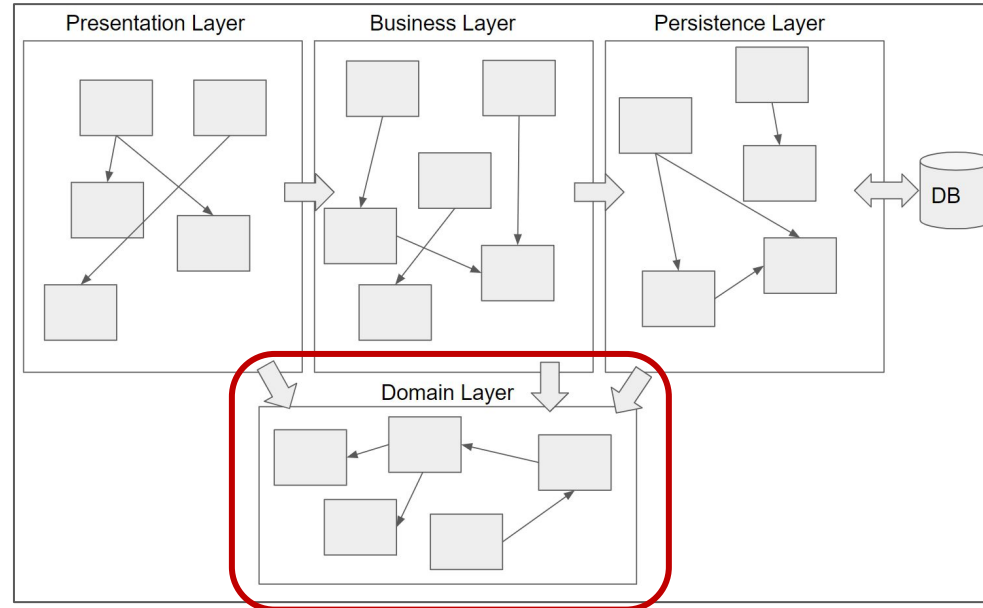
3 - Layered Architecture

- Split the application into 3 layers + the domain layer
- Roadmap for your code
- Provides
 - Overview
 - Reusability
 - Maintainability
 - ...

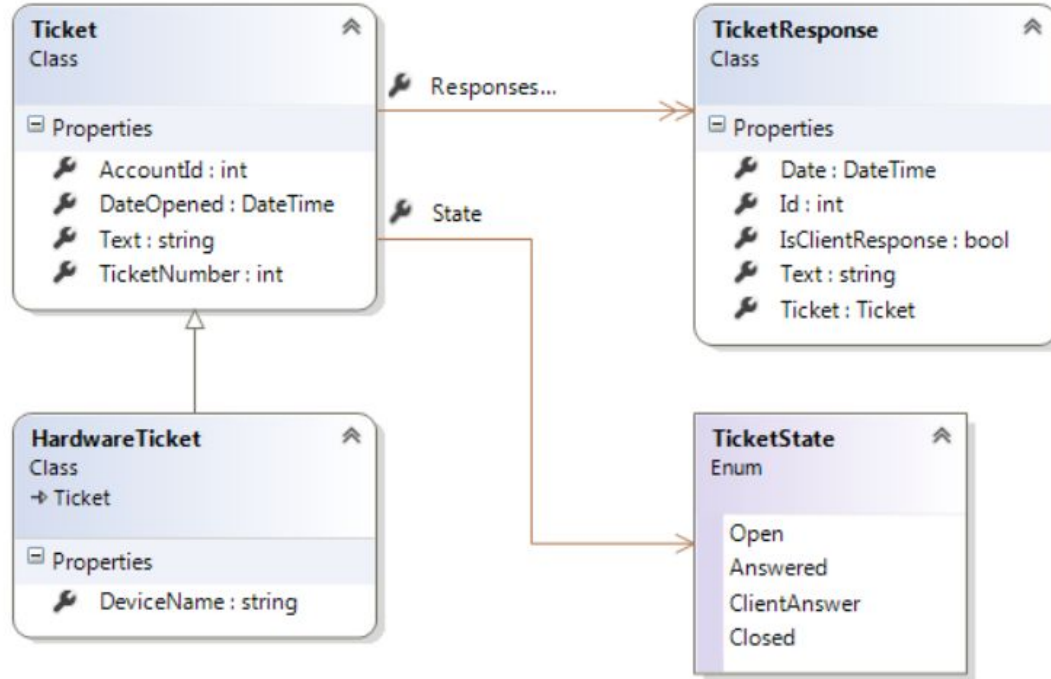


Domain Layer: based on the *Domain Model*

- The application is build around a 'Domain Model'
- It is the foundation of the application
- It is also called 'conceptual model'

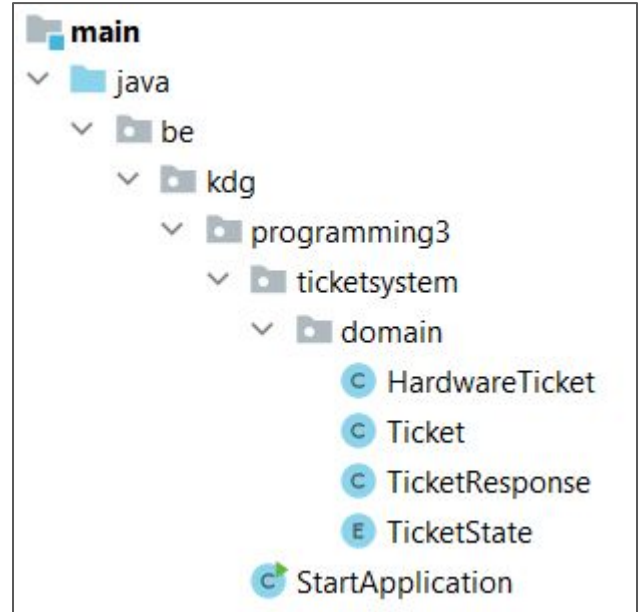


Domain Model: example (ticket system)



Domain Model → Domain Layer

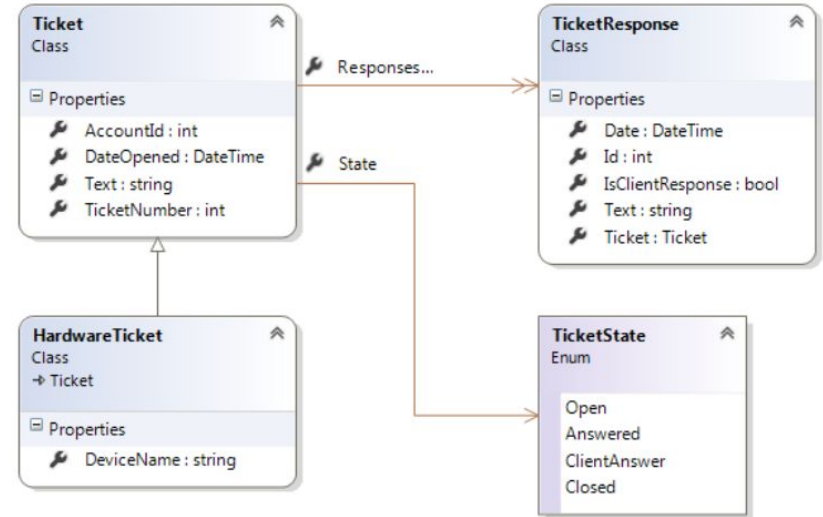
- Domain Model serves as a starting point for creating the classes in the *Domain Layer*
- Names of classes match the names of the entities of the Domain Model
- Relationships can translate into attributes (or extra classes)
- We create a java package **domain** to store the Domain Layer classes



Exercise 1: Domain Layer

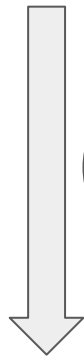


- Support Center: we will implement a support center
- The Domain Model is given...
- Implement the Domain Layer!
- Create some valid data (in main method):
 - Make 2 tickets
 - Add responses to the tickets
 - Show all tickets (using toString)
 - Show all responses of a ticket



N-Layer architecture: dependencies...

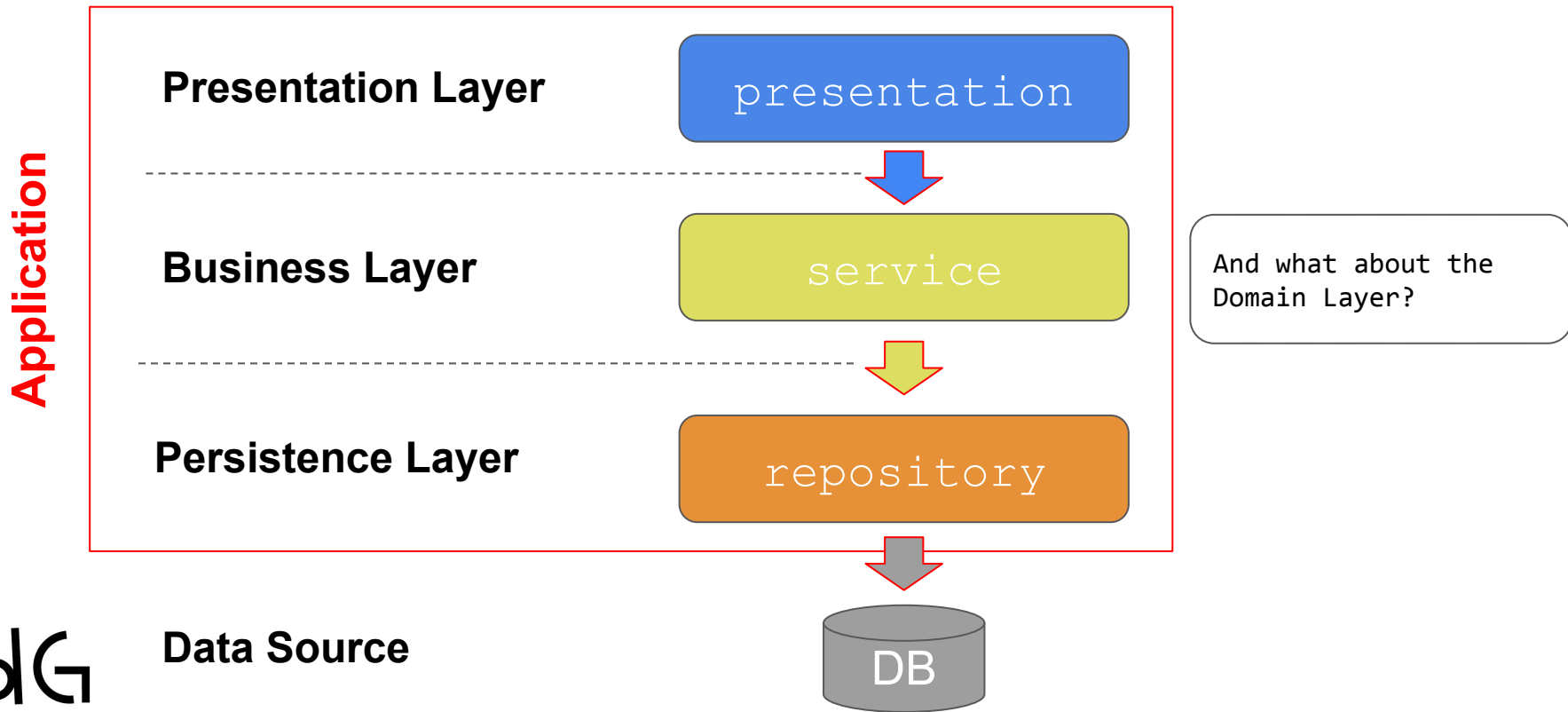
- Split up the application in logical components/layers
- Classical 3-layer model (+ domain layer):
 - **Presentation layer**
 - Visual representation
 - Interaction with the user
 - **Business Layer**
 - Domain models
 - Services
 - Business logic
 - **Persistence Layer**
 - Data access and data storage



A layer can only have a dependency on a lower layer!

N-Layer architecture: dependencies...

Java package:



Java package:

Application

Presentation Layer

presentation

Business Layer

service

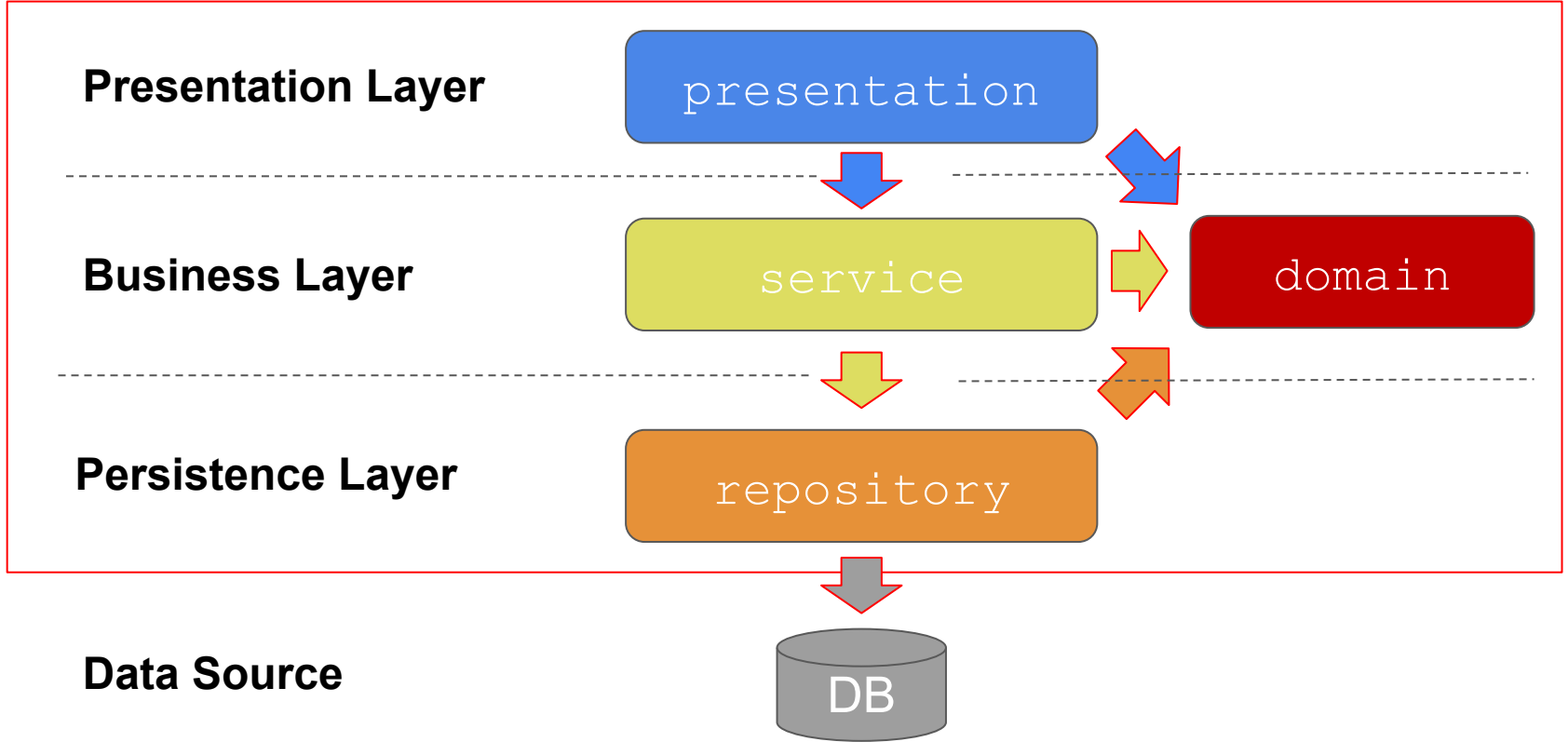
domain

Persistence Layer

repository

Data Source

DB



Demo 1: Layered Architecture

- In this demo we will
 - Create a small 3-layered console application (Student Management System)
 - i. Create the domain package: a Student has an id, name, birthdate and length
 - ii. Create the repository package with one repository class: StudentRepository. You can create and read students. We use a static List<Student> (represents the “Database”)
 - iii. Create the service package with one service class: StudentService. You can add students and get all students.
 - iv. Create the view package with one view class: View. It has a show method to show a menu to list all students and add a new student



Exercise 2: Layered Application (1/2)



- Upgrade the previous exercise (Support Center) to a small 3-layered application.
 - Create the repository package, it has one class TicketRepository:
 - Use a static List of Tickets.
 - Create a seed method to seed the lists with data. Run from a static block.
 - Provide 2 (non-static!) methods:
 - Ticket createTicket(Ticket ticket): incoming ticket gets a unique ticketnumber (highest + 1)
 - List<Ticket> readTickets(): returns current list of Tickets

See next slide for the rest of the exercise...

Exercise 2: Layered Application (2/2)



- Create the service package, it has one class `TicketService`. This class has a `TicketRepository` attribute, created in the constructor. It has the following methods:
 - `List<Ticket> getTickets()`: returns all tickets. Use the repository...
 - `Ticket addTicket(int accountId, String question)`: to create a Ticket. Use the repository...
 - `Ticket addTicket(int accountId, String device, String problem)`: to create a `HardwareTicket`. Use the repository...
- Create a presentation package, it has one class `View`. This class has two attributes: a `TicketService` and a `Scanner`. It has one public method:
 - `show()`: shows a small menu (0:exit, 1:show all tickets, 2: add ticket). This method loops until the user chooses exit.
 - Add functionality to show the tickets and be able to add a ticket
- Create a `View` from the main method and run the `show()` method



Agenda this week

Last week - project Review

3-Layer Architecture

Loose Coupling With Interfaces

Dependency Injection

@Annotations

Spring Framework: The Spring Container

Component Scanning - Autowiring

Loose coupling with Interfaces

- Can we make the dependencies *loosely coupled*?
 - It should be possible to switch to another implementation without affecting the dependent classes/layers
 - It should be possible to test classes/layers without the need for a full implementation of the classes/layers it depends on
- We can do this using interfaces
 - A class only talks to via an interface to its dependencies
 - The dependencies implement the interfaces

Demo 2: Loose Coupling with Interfaces

- In this demo we will
 - Demonstrate loose coupling with interfaces on the small Student Management System from previous demo
 - We extract an interface from the StudentRepository
 - The current class becomes an implementation of this interface (HardCodedStudentRepository)
 - We do the same for StudentService
 - We can provide a DummyStudentService implementation that only logs some output. That way we can test the View without using the real StudentService implementation



Exercise 3: Loose Coupling

- Refactor the Ticket System:
 - convert the repository and service classes into interfaces and use them
 - Provide some dummy implementations





Agenda this week

Last week - project Review

3-Layer Architecture

Loose Coupling With Interfaces

Dependency Injection

@Annotations

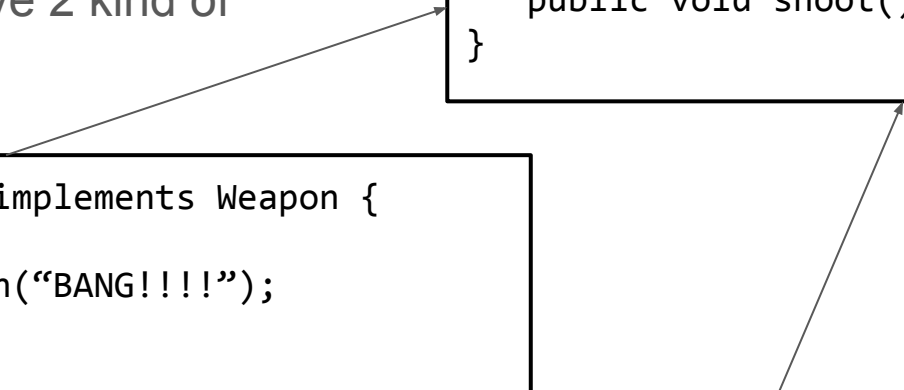
Spring Framework: The Spring Container

Component Scanning - Autowiring

Dependency Injection

- Example: we have 2 kind of Weapons:

```
public interface Weapon {  
    public void shoot();  
}
```



```
graph BT; LoudWeapon --> Weapon; QuietWeapon --> Weapon;
```

```
public class LoudWeapon implements Weapon {  
    public void shoot() {  
        System.out.println("BANG!!!!");  
    }  
}
```

```
public class QuietWeapon implements Weapon{  
    public void shoot() {  
        System.out.println("poef.");  
    }  
}
```

Dependency Injection

- Example: class `Monster` depends on `Weapon`.
- It could create the instance itself:
- **Problem:**
 - to switch to another `Weapon`, you have to alter the class `Monster`!

```
public class Monster{
    private final Weapon weapon;

    Monster() {
        weapon = new LoudWeapon();
    }

    public void attack() {
        for (int i=0;i<10;i++) {
            weapon.shoot();
        }
    }
}
```

Dependency Injection: solution

- Use interface Weapon (ok)
- **Inject the dependency** (eg. via constructor parameter)
- Class can stay the same!

```
new Monster(new LoudWeapon())
```

```
public class Monster{  
    private final Weapon weapon;  
  
    Monster(Weapon weapon) {  
        this.weapon = weapon;  
    }  
  
    public void attack() {  
        for (int i=0;i<10;i++) {  
            weapon.shoot();  
        }  
    }  
}
```

Demo 3: Dependency Injection

- In this demo we will
 - We will refactor our Student Management System:
 - We will use Dependency Injection now
 - We have to add some 'glue' code to construct the different object
 - We can now swap the implementation of the service class without changing the view class.



Exercise 4

- Refactor the Ticket System:
 - Use Dependency Injection
 - Add the 'glue' code in your StartApplication
 - Does it still run?
 - Swap the implementations with dummy implementations and run again



Remarks

- Working with N-layer architecture and using interfaces and dependency injection is a *design pattern* (a guideline), the goal is:
 - **Separation of concerns**
 - Each layer has its own layer-specific task, described in and accessed through interfaces
 - **Single Responsibility Principle**
 - Each class has small set of responsibilities
 - Business Layer and Persistence Layer can have multiple service- and repository classes.
 - Use extra (helper-)classes for extra functionality

Sidestep: where is MVP?

- Remember: Model - View - Presenter
 - Model: contains the data
 - View: only view code
 - Presenter: contains the view logic
- Model: we can use our Domain classes and Service layer
- View: No logic, only how we present data and actions on the screen
(`System.out.println, format...`)
- Presenter: contains the UI logic

Demo 4: Introduce MVP

- In this demo we will
 - We will refactor our Student Management System:
 - We will create a Presenter class in the presentation layer
 - Move all UI logic to this class
 - There is a dependency from the Presenter to the View
 - There is also a dependency from the View to the Presenter
 - It would be better if the View was agnostic about the Presenter
 - We could introduce the *Observer* design pattern here:

<https://www.baeldung.com/java-observer-pattern>

@Extra



Exercise 5

- Refactor the Ticket System:
 - Introduce MVP in your exercise
 - (Extra: try using the *observer* pattern to avoid a direct reference from the view to the presenter (not so easy...!))





Agenda this week

Last week - Project Review

3-Layer Architecture

Loose Coupling With Interfaces

Dependency Injection

@Annotations

Spring Framework: The Spring Container

Component Scanning - Autowiring

Annotations



- Labels you can add to your code
- Added since JDK5
- Can be used by the compiler, frameworks or tools
- Examples
 - `@Override`: the following method overrides a method. Compiler will now check if that is true.
 - `@Deprecated`: the following method should no longer be used. Compiler will now check if it is used and gives a warning.
 - `@SuppressWarnings`: suppress compiler warnings (for example: unused variable)
 - `@FunctionalInterface`: the following interface is a functional interface. Compiler checks if it has just one method.
- Java Code Geeks Annotation tutorial
(<https://www.javacodegeeks.com/2014/11/java-annotations-tutorial.html>)

Annotations are used by frameworks and libraries

- In the following lessons we will meet some frameworks and libraries that make use of their own annotations
- Examples
 - Spring: @Bean, @Component, @Controller, ...
 - GSON: @SerializedName, @Expose, ...
 - JUnit: @Test, @Before, @After, ...
 - ...

JUnit

{ GSON }



Annotations parameters

- No parameter: it's called a **Marker** annotation
 - example: `@Override`
- One parameter: **Single-value** annotation
 - Example: `@SuppressWarnings(value = "deprecation")`
 - If you use a single parameter and it is called "value" you can omit the name:
`@SuppressWarnings("deprecation")`
- More parameters: **Full** annotation
 - Example: `@Deprecated(since = "4.5", forRemoval = true)`
- Some parameters are arrays. Example:

```
@RequestMapping(  
    method = { RequestMethod.PUT, RequestMethod.POST },  
    value = "/ex/foos/multiple"  
)
```

This is also an array parameter!
But you can always pass a single
value as well {just drop the curly
braces}

Check the javadoc

All Classes and Interfaces

Interfaces

Classes

Enum Classes

Exception Classes

Annotation Interfaces

Class	Description
Deprecated	A program element annotated @Deprecated is one that programmer
Function annotation interface in java.lang	An informative annotation type used to indicate that an interface is a <i>functional interface</i> as defined by the Java Language Specification
Override	Indicates that a method declaration is intended to override a method
SafeVarargs	A programmer assertion that the body of the annotated method or method call does not perform potentially unsafe operations on its varargs parameter.
SuppressWarnings	Indicates the warnings to be suppressed at compile time in the annotated element.

Module java.base
Package java.lang

Annotation Interface Deprecated

```
@Documented
@Retention(RUNTIME)
@Target({CONSTRUCTOR, FIELD, LOCAL_VARIABLE, METHOD, PACKAGE, MODULE, PARAMETER, TYPE})
public @interface Deprecated
```

Element Details

since

String since

Returns the version in which the annotated element became deprecated. The version string is in the same format and namespace as the value of the @since javadoc tag. The default value is the empty string.

Returns:
the version string

Since:
9

Default:
""

forRemoval

boolean forRemoval

Indicates whether the annotated element is subject to removal in a future version. The default value is false.

Returns:
whether the element is subject to removal

Since:
9

Default:
false

Create your own Annotations?

- You can write your own annotations
- For example: you make a performance testing framework that will run all methods in a class which have the annotation `@Heavy`
 - The annotation will have one parameter: the maxtime the method can take
 - Your framework will run all the `@Heavy` methods and report the ones that took more time than the maxtime



Create your own Annotations

The annotation has annotations itself!

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
public @interface Heavy {
    int maxTime() default 100;
}
```

It is a single-value annotation with a default value (the user can omit the annotation)

`@Retention` determines at what point annotation should be discarded.

- SOURCE will be retained only with source code, and discarded during compile time.
- CLASS will be retained till compiling the code, and discarded during runtime.
- RUNTIME will be available to the JVM through runtime.

`@Target` determines where the annotation can be used. Examples are: FIELD, METHOD, CONSTRUCTOR...

Use your own Annotation

```
public class MyHeavyMethodsClass {  
    @Heavy(maxTime = 100)  
    public static void slowMethod(){  
        System.out.println("slow method starting");  
        try {  
            Thread.sleep(new Random().nextInt(500));  
        } catch (InterruptedException e) {  
            e.printStackTrace();  
        }  
    }  
  
    public static void normalMethod(){  
        System.out.println("This is a normal method");  
    }  
}
```



Our little “framework” code...

```
for (Method method: MyHeavyMethodsClass.class.getDeclaredMethods()) {  
    Heavy heavy = method.getAnnotation(Heavy.class);  
    if (heavy!=null) {  
        System.out.println("Found heavy method:" + method.getName());  
        try {  
            long time = System.currentTimeMillis();  
            method.invoke(null);  
            long deltaTime = System.currentTimeMillis() - time;  
            if (deltaTime>heavy.maxTime()) {  
                System.out.println("The method was to slow!");  
                System.out.printf("It took %d ms, while maxtime was %d ms!\n",  
deltaTime, heavy.maxTime());  
            }  
        } catch (IllegalAccessException | InvocationTargetException e) {  
            e.printStackTrace();  
        }  
    }  
}
```

This code uses the Java Reflection API.
This API is not part of this course...

@Extra

demo_annotations

- In this demo we will
 - Create a Heavy annotation, it is Single-valued
 - Use this annotation on some methods
 - Use reflection to read out the annotation and run the methods



Exercise 6



- Create a new gradle project.
- Write an annotation `@NotFinished`. It should be used by developers to label methods that they have not yet finished.
- Use it in a demo class `Demo` that has at least 4 methods. Some are `@NotFinished`.
- Write a `StartApplication` class that uses reflection to make a small report of all the `@NotFinished` methods in the `Demo` class.
- Add a parameter to the Annotation: `developer` (a `String`).
- Add the developers name to the corresponding unfinished methods in your report.



Agenda this week

Last week - project Review

3-Layer Architecture

Loose Coupling With Interfaces

Dependency Injection

@Annotations

Spring Framework: The Spring Container

Component Scanning - Autowiring

What is Spring?

- Spring is an open source framework to build *enterprise applications* in Java
- Main goal: Simplify Java Development
- Spring consists of many *projects*: <https://spring.io/projects>
 - **Spring Framework**
 - Spring Boot
 - Spring Data
 - Spring Cloud
 - Spring Security
 - Spring GraphQL
 - Spring Session
 - Spring Integration
 - ...



→ Each of the projects has its own version

What is the Spring Framework?

- **Spring Container with Dependency Injection**
- Web and remoting (Spring MVC, Webflux, ...)
- Data Access and Integration
 - Less boilerplate JDBC with templates
 - Object Relational Mapping
- **Aspect Oriented Programming:**
 - implement application wide concerns
- Messaging
- Instrumentation: measure performance, diagnose errors,...
- Testing
- ...

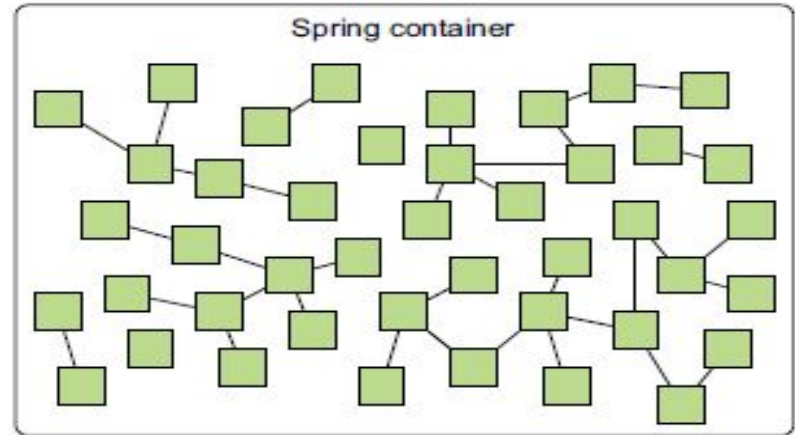
Spring Container with Dependency Injection

Enterprise application: is composed of many components that need to work together. Each component is responsible for a part of the functionality.

Components need to be created and work together

Spring Container (“Spring Application Context”) creates and manages these components (or *beans*)

This is done by using *Dependency Injection*



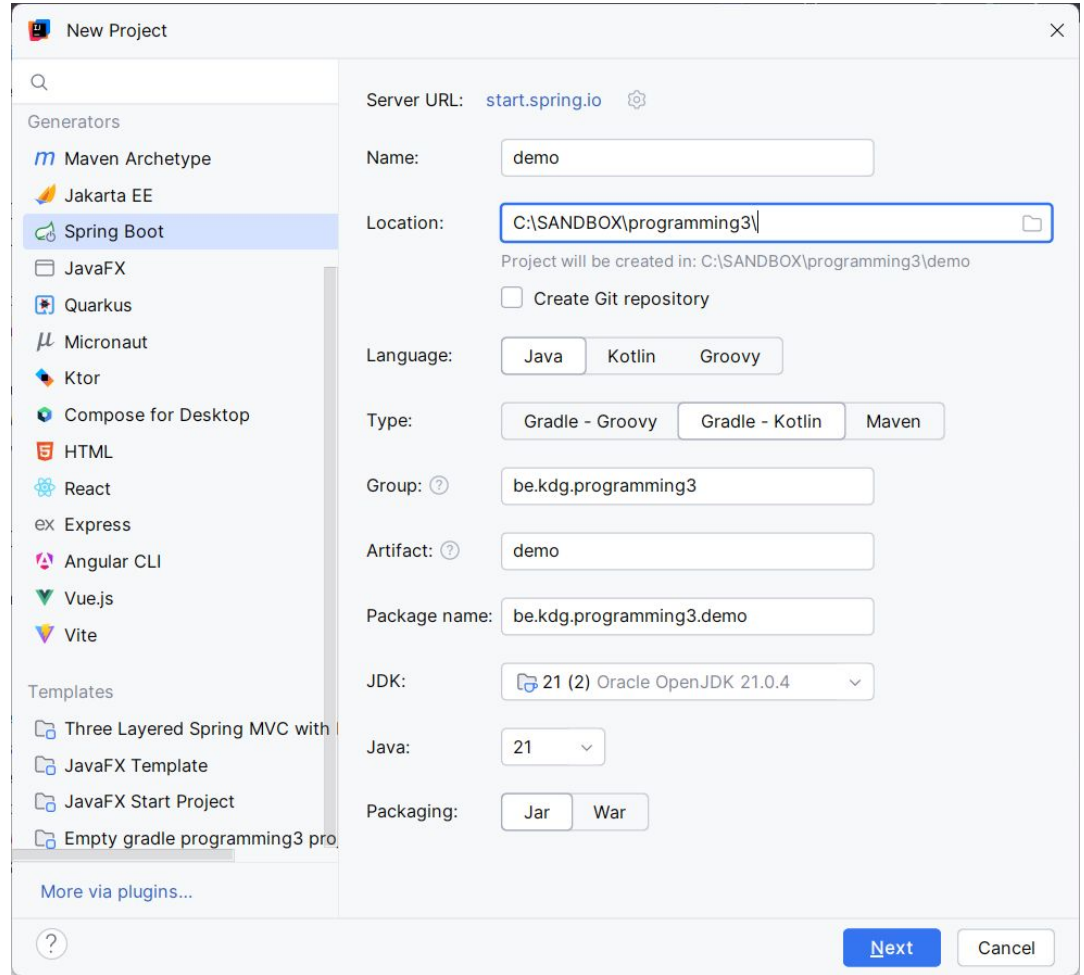
Dependency Injection Frameworks

- Also called Dependency Injection Containers
- Are in fact an implementation of the Abstract Factory design pattern
- Spring is not the only DI Framework
 - Enterprise Java Beans
 - Dagger (Android)
 - Micronaut
 - Quarkus
 - ...



IntelliJ Spring project

- File → new → Spring Boot
 - Gradle project
- No extra Dependencies (for now :)



build.gradle.kts

```
plugins {  
    java  
    id("org.springframework.boot") version "3.3.4"  
    id("io.spring.dependency-management") version "1.1.6"  
}  
//...  
dependencies {  
    implementation("org.springframework.boot:spring-boot-starter")  
    testImplementation  
        ("org.springframework.boot:spring-boot-starter-test")  
}  
//...
```

Example: Monsters and Weapons

The Weapon interface is injected in the ScaryMonster

- We created a small demo project that uses DI:

```
public interface Monster{  
    void attack();  
}
```

```
public interface Weapon{  
    void shoot();  
}
```

```
public class ScaryMonster implements Monster{  
    private Weapon weapon;  
  
    public ScaryMonster(Weapon weapon) {  
        this.weapon = weapon;  
    }  
  
    @Override  
    public String attack() {  
        System.out.println("GRRRRR!!!");  
        weapon.shoot();  
    }  
}
```

```
public class LoudWeapon  
implements Weapon{  
    @Override  
    public void shoot() {  
        sout("BANG!!!");  
    }  
}
```

The 'glue' code: making a weapon and a monster

```
Weapon weapon = new LoudWeapon();  
Monster monster = new ScaryMonster(weapon);  
monster.attack();
```

Can Spring create the objects for us?

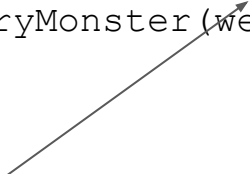
The Weapon implementation is injected in the ScaryMonster implementation

Configure the container in @Configuration class

- @Configuration marks the configuration class
 - Configuration can also be done using XML files...
- Best delivered as source code → should be easy to alter
- @Bean annotation: marks the factory method for a bean.
 - You don't call this method, it is configuration!
- Spring sees a Weapon parameter is needed → Spring will look for the first bean that implements the interface and inject it

```
@Configuration
public class MonsterConfiguration {
    @Bean
    public Monster monster(Weapon weapon){
        return new ScaryMonster(weapon);
    }

    @Bean
    public Weapon weapon(){
        return new LoudWeapon();
    }
}
```



Using the @Beans

- @SpringBootApplication: class to start the application
- ConfigurableApplicationContext is the container that contains the beans and is managed by the Spring Framework
- Beans are by default Singleton: only one instance of the bean exists

```
@SpringBootApplication
public class SpringInjectionApplication {
    public static void main(String[] args) {
        ConfigurableApplicationContext context =
            SpringApplication.run(MonsterConfiguration.class, args);
        context.getBean(Monster.class).attack();
        context.close();
    }
}
```

What classes are @Bean candidates?

- Classes based on interfaces with possibly multiple implementations
 - Services, repositories, ...
- You can use this configuration file to choose the correct implementation



Exercise 7

- Create the small Monster application that we showed in the slides and experiment!



Demo: Use Spring on Student System

- In this demo we will
 - Look at our previous demo application: the Student Management System
 - We will create a Spring application that uses the application context to create the Beans and run the application!



Exercise 8

- Now try to create a Spring application for the Support System!





Agenda this week

Project Review

3-Layer Architecture

Loose Coupling With Interfaces

Dependency Injection

@Annotations

The Spring Container

Component Scanning - Autoconfiguration

Component scanning - Autowiring

- It is also possible to let Spring create and inject beans without a `@configuration` class!
- Instead of using `@Bean` in a `@Configuration` class, you can use `@Component` in the implementation class

```
@Component  
public class LoudWeapon  
implements Weapon{  
    @Override  
    public void shoot() {  
        //...  
    }  
}
```

Component scanning - Autowiring

- **@Autowired** tells Spring to find an implementation of Weapon and inject it!
 - You can also use **@Inject**

We'll see later that on the **constructor** of a component autowiring even works without the **@Autowired** annotation

```
@Component
public class ScaryMonster implements Monster
{
    private Weapon weapon;

    @Autowired
    public ScaryMonster(Weapon weapon) {
        this.weapon = weapon;
    }

    @Override
    public String attack() {
        System.out.println("GRRRRR!");
        weapon.shoot();
    }
}
```

Component scanning - Autowiring

- We no longer need the `@Configuration` class
- Spring will scan for `@Component` classes in **packages and subpackages** starting from the location of the `@SpringBootApplication` class.
- You can combine component scanning and a `@Configuration` class

```
@SpringBootApplication
public class SpringInjectionApplication {
    public static void main(String[] args) {
        ConfigurableApplicationContext context =
            SpringApplication.run(SpringInjectionApplication.class, args);
        context.getBean(Monster.class).attack();
        context.close();
    }
}
```

What if there is more than one implementation?

- If there is more than one implementation of the interface, which one do we use?
 - Use the `@Qualifier` annotation!
 - Add it to the `@Component` annotation to give the component a name

```
@Qualifier("loud")  
@Component  
public class LoudWeapon implements Weapon {  
    @Override  
    public void shoot() {  
        //loud bang..!  
    }  
}
```

```
@Qualifier("quiet")  
@Component  
public class QuietWeapon implements Weapon {  
    @Override  
    public void shoot() {
```

```
@Component  
public class ScaryMonster implements Monster{  
    private Weapon weapon;  
  
    public ScaryMonster(@Qualifier("loud") Weapon weapon)  
        this.weapon = weapon;  
}
```

Exercise 9

- Refactor your previous exercise: use component scanning and autowiring!



Running a commandline Spring application 1/2

- If a `@Component` that implements `CommandLineRunner`, Spring will automatically invoke it's run method on startup

```
@SpringBootApplication
public class SpringConsoleApp {
    public static void main(String[] args) {
        SpringApplication.run(SpringConsoleApp.class, args);
    }
}
```

Running a commandline Spring application 2/2

```
@Component
public class MonsterRunner implements CommandLineRunner {
    private final Monster monster;

    public MonsterRunner(Monster monster) {
        this.monster = monster;
    }

    @Override
    public void run(String... args) {
        System.out.println("Let the monster attack!");
        monster.attack();
    }
}
```

Exercise 10 (1/3)



- If you have the Support System up and running with component scanning, try to add complete CRUD (Create, Read, Update, Delete) functionality to it:
 - In the repository:
 - `Ticket readTicket(int ticketNumber)`
 - `void updateTicket(Ticket ticket)`
 - `void deleteTicket(int ticketNumber)`
 - `List<TicketResponse> readTicketResponsesOfTicket(int ticketNumber)`
 - `TicketResponse createTicketResponse(TicketResponse response)`

→ Tip: `deleteTicket` also removes the responses of the concerning ticket, `createTicketResponse` creates an id.

Exercise 10 (2/3)



- In the business layer: TicketService
 - Ticket getTicket(int ticketNumber)
 - void changeTicket(Ticket ticket)
 - void removeTicket(int ticketNumber)
 - List<TicketResponse> getTicketResponses(int ticketNumber)
 - TicketResponse addTicketResponse(int ticketNumber, String response, boolean isClientResponse)

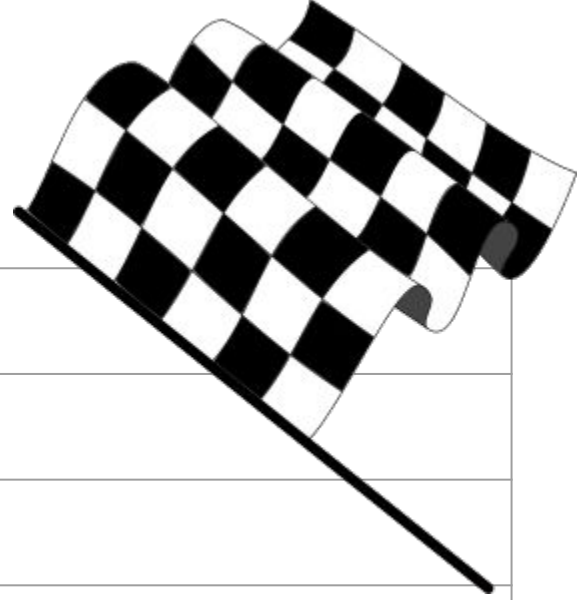
→ Tip: addTicketResponse adds a response to the Ticket but also updates the status of the Ticket using isClientResponse

Exercise 10 (3/3)

- In the presentation layer
 - Expand the menu:
 1. Show all tickets
 2. Show details of ticket
 3. Show answers of ticket
 4. Make a new ticket
 5. Answer a ticket
 6. Remove a ticket
 7. Exit
 - Implement all functionality!



Agenda this week



Project Review
3-Layer Architecture
Loose Coupling With Interfaces
Dependency Injection
@Annotations
The Spring Container
Component Scanning - Autoconfiguration

Project

You refactor the application, it should use a **layered architecture**:

- presentation layer: contains the view code. Try to implement MVP here: the view code is separated from the view logic.
- business layer (services + domain): use two packages, implement a Service class for each of the two main entities.
- data access layer: contains the data classes (repositories).

Add **interfaces** to provide loose coupling between the layers. Use dependency injection to connect the different classes.

And finally convert it to a **Spring application** using the Spring Boot template. Use **component scanning** and **autowiring** to connect the components and configure the application context. Use a commandlinerunner to start the application.

Push your results to a private repository on gitlab, tag it as 'sprint1' and submit the URL of your repository as a response to the canvas assignment. Be sure to add your teacher as Reporter to your project!

